

PP_T2_2023-01-08

[📄: PP Test 2 2023-01-08 OCRd, p.1](#)

Aufgabe 1 (30 Punkte)

Bitte markieren Sie jedes Auswahlfeld, bei dem die links stehende Aussage eine Eigenschaft des darüber stehenden Entwurfsmusters ist. Es können keines, eines oder mehrere Felder pro Zeile auszuwählen sein.

Eigenschaft	Iterator	Decorator	Proxy	Prototype	Factory-M.	Erklärung
beruht auf <i>Delegation</i>		x	x			Beide leiten Aufrufe an ein anderes Objekt weiter (Decoratee bzw. RealSubject)
verwendet häufig <i>Hooks</i>					x	Die Factory-Methode selbst ist oft ein Hook, den Unterklassen überschreiben.
<i>Subject</i> ist ein Bestandteil			x			<i>Subject</i> (Subjekt) ist Teil des Proxy-Musters (definiert die Schnittstelle für Proxy und RealSubject).
<i>Creator</i> ist ein Bestandteil					x	<i>Creator</i> und <i>ConcreteCreator</i> sind die zentralen Klassen der Factory-Methode.
<i>Product</i> ist ein Bestandteil					x	Das, was vom Creator erzeugt wird.
wird auch <i>Wrapper</i> genannt		x				Decorator "umwickelt" ein Objekt. (Adapter wird auch so genannt).
wird auch <i>Surrogate</i> genannt			x			Surrogate (Stellvertreter) ist Synonym für Proxy.
<i>Component</i> ist ein Bestandteil		x				<i>Component</i> ist das Interface Decorator-Pattern. Proxy nutzt meist den Begriff <i>Subject</i> .
ist erzeugendes Entwurfsmuster				x	x	Prototype und Factory dienen der Objekterstellung.
ist Entwurfsmuster für Struktur		x	x			Decorator und Proxy befassen sich mit der Zusammensetzung von Klassen/Objekten.
führt zu vielen kleinen Objekten		x				Decorator führt oft zu vielen kleinen, ähnlich aussehenden Wrapper-Objekten.

Eigenschaft	Iterator	Decorator	Proxy	Prototype	Factory-M.	Erklärung
auch auf leeren <i>Aggregaten</i> sinnvoll	x					Ein Iterator muss auch leere Listen handhaben können (<code>hasNext()</code> liefert sofort <code>false</code>).
<i>robuste</i> Varianten werden bevorzugt	x					Robuste Iteratoren erlauben Änderungen an der Liste während der Iteration.
flache und tiefe Kopien unterschieden				x		Das Kernproblem beim Prototyping (Cloning).
es gibt interne und externe Varianten	x					Intern: Iterator steuert selbst Extern: Client steuert (z.B. Java Iterator).
führt zu parallelen Klassenhierarchien					x	Für jedes <i>ConcreteProduct</i> w oft ein spezifischer <i>ConcreteCreator</i> benötigt.
häufig als innere Klasse implementiert	x					Iteratoren benötigen Zugriff private Felder des Aggregats daher oft <code>inner class</code> .
Objektidentität ist damit unzuverlässig		x				Der Client arbeitet mit dem Wrapper, nicht dem Original (<code>wrapper != original</code>).
oft große Anzahl an Unterklassen nötig					x	Siehe "parallele Klassenhierarchien". Decorator <i>vermeidet</i> dies eher.
wird auch <i>Virtual-Constructor</i> genannt					x	Synonym für Factory-Methode
zyklische Strukturen bereiten Probleme				x		Beim tiefen Kopieren (Cloning) führen Zyklen zu Endlosschleifen
<i>Smart-Reference</i> ist eine Variante davon			x			Ein Proxy, der zusätzliche Aktionen beim Zugriff ausführt (z.B. Referenzzähler).
auch für kovariante Probleme einsetzbar					x	Factory erlaubt Rückgabe von spezifischeren Unterklassen (Kovarianz der Rückgabotypen)
hilft große Zahl an Klassen zu vermeiden		x				Statt <code>TextMitRahmenUndScroll</code> (Vererbungsexplosion) kombiniert man Decorators dynamisch.

Eigenschaft	Iterator	Decorator	Proxy	Prototype	Factory-M.	Erklärung
für oberflächliche Erweiterungen geeignet		x				Decorator fügt Verhalten ("Schmuck") hinzu, ohne die Klasse zu ändern.
auch für entziehbare Verantwortlichkeiten		x				Decorators können theoretisch entfernt oder anders geschachtelt werden.
schlecht geeignet für umfangreiche Objekte		x				Hat die Komponente sehr viele Methoden, muss der Decorator alle durchleiten (viel Boilerplate).
kann gleiche Struktur wie <i>Decorator</i> haben			x			Proxy und Decorator haben fast identische Klassenstrukturen (Wrapper + Interface).
unterstützt Umkehrung der Abhängigkeiten					x	Factory entkoppelt die Instanziierung -> Client hängt vom Interface (Product), nicht von der Klasse ab.
mehrere gleichzeitige Abarbeitungen möglich	x					Mehrere Iteratoren können gleichzeitig auf derselben Liste operieren.

Aufgabe 2 (10 Punkte)

Bitte markieren Sie jedes Auswahlfeld, bei dem die links stehende Aussage eine Eigenschaft des darüber stehenden Entwurfsmusters ist. Es können keines, eines oder mehrere Felder pro Zeile auszuwählen sein.

Eigenschaft	Factory-M.	Template-M.	Visitor	Singleton	Erklärung
Methode <code>accept</code> kommt vor			x		Zentraler Bestandteil des Double-Dispatch (<code>element.accept(visitor)</code>).
Anzahl der Objekte kontrollierbar				x	Garantiert genau eine Instanz (oder eine festgelegte Anzahl).
häufig als <i>Anti-Pattern</i> angesehen				x	Wegen globalem State, schlechter Testbarkeit und versteckten Abhängigkeiten.
<i>Hollywood-Prinzip</i> spielt eine Rolle	x	x			"Don't call us, we'll call you." Die abstrakte Oberklasse ruft die Methoden der Unterklasse auf (Inversion of Control).
sehr viele Klassen können entstehen	x				Durch parallele Hierarchien (für jedes Produkt ein Creator).
sehr viele Objekte können entstehen					(Trifft eher auf Decorator oder Flyweight-Szenarien zu).
sehr viele Methoden können entstehen			x		Wenn die Objektstruktur viele Klassen hat, muss der Visitor für <i>jede</i> Klasse eine <code>visit</code> -Methode haben.
verwandte Operationen zentral verwaltet			x		Visitor trennt Algorithmen von Daten. Alle Operationen eines Typs (z.B. "Export") landen in <i>einer</i> Visitor-Klasse statt verteilt in den Datenklassen.
kann mit kovarianten Problemen umgehen	x				Erlaubt in Unterklassen die Rückgabe spezialisierterer Produkttypen (kovarianter Rückgabetyt).

Eigenschaft	Factory-M.	Template-M.	Visitor	Singleton	Erklärung
<i>Hooks</i> von abstrakten Methoden unterschieden		x			Unterscheidung zwischen <i>muss</i> überschrieben werden (abstrakt) und <i>kann</i> (Hook mit leerer/default Impl).

Aufgabe 3 (10 Punkte)

Bitte markieren Sie pro Zeile das eine Auswahlfeld, bei dem die links stehende Aussage am ehesten eine Eigenschaft der darüber stehenden Parametrisierungsform in Java oder AspectJ ist.

Eigenschaft	Annotationen	Aspekte	Generizität	Erklärung
<code>T+</code> steht für jeden Untertyp eines Typs <code>T</code>		x		AspectJ Pointcut-Syntax: <code>T</code> matcht <code>T</code> und alle Subtypen
dynamisch nur über <i>Reflexion</i> verwendbar	x			Annotationen mit <code>RetentionPolicy.RUNTIME</code> werden mittels Reflection ausgelesen.
drei Arten von <i>Advice</i> werden unterschieden		x		Before, After (returning/throwing) und Around Advice.
gebundene <i>Wildcards</i> erhöhen die Flexibilität			x	Java Generics: <code>? extends T</code> (Upper Bound) und <code>? super T</code> (Lower Bound).
ideal geeignet für <i>Querschnittsfunktionalitäten</i>		x		"Cross-cutting concerns" (z.B. Logging, Security) sind der Kern von AOP.
<code>args(...)</code> hilft beim Sammeln von Kontextinformation		x		<code>args()</code> ist ein AspectJ Pointcut-Designator, um Argumente an Advice-Parameter zu binden.
implizite Untertypbeziehungen werden nicht unterstützt			x	Invarianz: <code>List<String></code> ist <i>kein</i> Untertyp von <code>List<Object></code> .
Typkonsistenz wird nur statisch durch <i>Typinferenz</i> überprüft			x	Type Erasure: Zur Laufzeit sind generische Typen weg (erased), Prüfung erfolgt nur zur Compilezeit.
über eine <code>default</code> -Klausel lassen sich <i>Default</i> -Werte angeben	x			Syntax: <pre>@interface MyAnno { String value() default "text"; }</pre>
binäre Methoden sind über rekursive Deklarationen ausdrückbar			x	F-Bounded Polymorphism: <pre>class Enum<E extends Enum<E>></pre> (Vergleich mit selbst).

Aufgabe 4 (10 Punkte)

Bitte markieren Sie pro Zeile das eine entsprechende Auswahlfeld, bei dem die links stehende Aussage im Zusammenhang mit Softwareentwurfsmustern (kurz *Muster* genannt) immer, manchmal oder nie zutrifft.

Aussage	immer	manchmal	nie	Erklärung
die Programmstruktur zeigt, um welches Muster es sich handelt		x		Struktur ist oft mehrdeutig (z.B. haben State & Strategy oder Proxy & Decorator fast identische Klassendiagramme). Die <i>Intention</i> entscheidet.
Muster können Ideen hinter dem Softwareentwurf sehr gut ersetzen			x	Muster sind Werkzeuge/Vokabular zur Umsetzung, ersetzen aber nicht das Nachdenken über Domänenlogik und Architektur.
Muster sollen zur Einschätzung von Konsequenzen eingesetzt werden	x			Das ist einer der Hauptzwecke von Mustern: Sie dokumentieren explizit <i>Trade-offs</i> (Vor- und Nachteile).
Muster helfen dabei, persönliche Expertisen auf Konsistenz zu prüfen	x			Durch Abgleich der eigenen Lösung mit etablierten Standards ("Habe ich das Rad neu erfunden oder ist das ein Visitor?").
der Begriff <i>Anti-Pattern</i> ist nur bei nicht-wertender Sichtweise sinnvoll			x	Ein Anti-Pattern ist <i>per Definition</i> negativ wertend (schlechte Lösung). Ohne Wertung wäre es einfach nur ein Muster.
der Einsatz von Mustern ist wichtiger als die Einhaltung von Konventionen			x	Code-Konventionen und Lesbarkeit (KISS) gehen vor. Erzwingen von Mustern führt zu unwartbarem Code.
Software ist besser, wenn mehr Muster und weniger <i>Anti-Pattern</i> vorkommen		x		Weniger Anti-Patterns: Ja. Aber <i>mehr</i> Muster heißt oft Over-Engineering ("Pattern Happy"). Die richtige Dosis entscheidet.
um Konsequenzen zu gewichten, sind wertende Sichtweisen von Mustern nötig			x	Begründung aus dem Skript: Um Konsequenzen zu gewichten, müssen wir Muster "fast zwangsläufig <i>nicht wertend</i> betrachten".
Konsequenzen eines Musters treffen auf alle Implementierungen des Musters zu		x		Kern-Konsequenzen treffen oft zu, aber spezifische (z.B. Performance) hängen stark von der konkreten Implementierung ab.

Aussage	immer	manchmal	nie	Erklärung
etablierte Muster sind zuverlässig, da lange Diskussionsprozesse dahinterstehen		x		Begründung aus dem Skript: Es heißt wörtlich: "Wie Faustregeln bieten auch Softwareentwurfsmuster keine zuverlässigen Aussagen , sondern nur zufallsbehaftete Entscheidungsgrundlagen."

Aufgabe 5 (15 Punkte)

Bitte markieren Sie jedes Auswahlfeld, bei dem die links stehende Kommandozeile, in `bash` fehlerfrei ausgeführt, die darüber stehende Auswirkung hat. Dabei sind die Auswirkungen folgendermaßen beschrieben:

- **back**: mindestens ein Prozess wird im Hintergrund gestartet
- **pipe**: mindestens eine Pipeline wird angelegt
- **args**: mindestens ein Kommandozeilenargument wird übergeben
- **in**: mindestens eine Standardeingabe wird zu einer Datei (nicht Pipeline) umgeleitet
- **out**: mindestens eine Standardausgabe wird zu einer Datei (nicht Pipeline) umgeleitet
- **err**: mindestens eine Fehlerausgabe wird zu einer Datei (nicht Pipeline) umgeleitet

Es können keines, eines oder mehrere Felder pro Zeile auszuwählen sein.

Kommandozeile	back	pipe	args	in	out	err	Erklärung
cat a			x				<code>a</code> ist ein Argument für <code>cat</code> .
cat a >> b			x		x		<code>>></code> hängt die Ausgabe an Datei <code>b</code> an (Out). <code>a</code> ist Argument.
cat < a wc > b		x		x	x		<code><</code> liest aus Datei (In), <code> </code> ist die Pipeline, <code>></code> schreibt in Datei (Out).
cat *.java cat b			x				<code>*.java</code> wird zu Dateinamen (Argumenten) expandiert. <code> </code> ist ein logisches ODER (keine Pipe).
cat a &> b			x		x	x	<code>&></code> leitet Standard- (Out) und Fehlerausgabe (Err) in Datei <code>b</code> um.
cat wc &	x	x					<code> </code> ist Pipeline, <code>&</code> am Ende startet im Hintergrund (Back).
cat wc &> a		x			x	x	Pipeline (<code> </code>) vorhanden. <code>&></code> leitet Out & Err von <code>wc</code> in Datei <code>a</code> .
cat a &>> b &	x		x		x	x	<code>&</code> (Back), <code>&>></code> (Out & Err append), <code>a</code> (Args).
cat < a &>> b				x	x	x	<code><</code> (In), <code>&>></code> (Out & Err append). Keine Argumente.
cat a && cat b			x				<code>&&</code> ist logisches UND. <code>a</code> und <code>b</code> sind Argumente.
cat a wc > b &	x	x	x		x		<code>&</code> (Back), <code> </code> (Pipe), <code>a</code> (Arg für cat), <code>></code> (Out für wc).

Kommandozeile	back	pipe	args	in	out	err	Erklärung
cat < a & wc > b &	x	x		x	x		& (Back), (Pipe für Out+Err), < (In), > (Out). Err geht in Pipe, nicht Datei.
if test a = a ; then cat < a ; fi			x	x			test a = a nutzt Argumente. cat < a nutzt Input-Umleitung.
for i in a b c ; do cat \$i ; done			x				cat \$i ruft cat mit den Argumenten a, b, c auf.
for i ... do (cat \$i > \$i.bak &) ...	x		x		x		& in der Schleife (Back), \$i (Args), > (Out).

Aufgabe 6 (13 Punkte)

Bitte markieren Sie jedes Auswahlfeld, bei dem der links stehende Typausdruck (mit Typnamen aus den Paketen `java.util.function` und `java.lang`) ein Typ des darüber stehenden Lambdas ist. Es können keines, eines oder mehrere Felder pro Zeile auszuwählen sein.

Typausdruck	()- >7L	s- >s+1	(s,t)- >s+t	s->t- >s+t	Long::sum
<code>BiFunction<Long, Long, Long></code>			x		x
<code>BiFunction<Long, String, Long></code>					
<code>BiFunction<Long, String, String></code>			x		
<code>BiFunction<String, Long, String></code>			x		
<code>BiFunction<String, String, String></code>			x		
<code>Function<Long, Function<Long, Long>></code>				x	
<code>Function<String, Function<Long, String>></code>				x	
<code>LongSupplier</code>	x				
<code>LongBinaryOperator</code>			x		x
<code>LongUnaryOperator</code>		x			
<code>Supplier<String></code>					
<code>BinaryOperator<String></code>			x		
<code>UnaryOperator<String></code>		x			

Aufgabe 7 (12 Punkte)

Beantworten Sie folgende Fragen bitte mit je drei Ausdrücken oder Begriffen. Vermeiden Sie Umschreibungen, Beschreibungen oder Erklärungen. Es werden jeweils nur die ersten drei Ausdrücke oder Begriffe gewertet.

1. Geben Sie bitte drei *rekursive Typparameterdeklarationen* an, also Ausdrücke, mit denen in Java rekursive Typparameter deklariert werden können. Sie sollen verschiedene Arten *gebundener Wildcards* enthalten:

- `<A extends Comparable<? super A>>`,
- `<A extends Comparable<? extends A>>`,
- `<A extends HashMap<? super A>>`

2. Geben Sie bitte drei verschiedene Namen von in Java standardmäßig vordefinierten Annotationen an. Achten Sie dabei bitte auf Java-konforme Groß- und Kleinschreibung:

- `@FunctionalInterface`
- `@Deprecated`
- `@Override`

3. Geben Sie bitte drei verschiedene Namen von Methoden an, die in

`java.util.stream.Stream<T>` deklariert sind und zu den *Stream-abschließenden Operationen* zählen:

- `collect`
- `forEach`
- `count`

4. Geben Sie bitte drei verschiedene Namen von Methoden an, die in

`java.util.stream.Stream<T>` deklariert sind und zu den *Stream-modifizierenden Operationen* zählen:

- `map`
- `filter`
- `sorted`